

PE-EC802C — VLSI Design Automation

Semester VIII • 3L:0T:0P • 3 Credits

Module 2: Layout Compaction, Placement & Partitioning

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1	Layout Compaction	2
1.1	1D vs. 2D Compaction	2
1.2	Constraint Graph Compaction	2
2	Circuit Placement	2
2.1	1. Force-Directed Placement	3
2.2	2. Min-Cut Placement (Breuer's Algorithm)	3
3	Circuit Partitioning	3
3.1	Kernighan-Lin (KL) Algorithm	3
3.1.1	Mathematical Formulations	3
3.1.2	Algorithmic Steps for One Pass	4
3.2	Fiduccia-Mattheyses (FM) Partitioning Algorithm	5
4	Quick Revision Page	7
4.1	Key Equations & Formulations	7
4.2	One-Line Definitions	7
4.3	Comparison: Algorithms & Compaction	7
4.4	Frequently Asked Questions / Viva Questions	9

1. Layout Compaction

Layout Compaction

Layout Compaction is the process of compressing a VLSI layout in the horizontal and/or vertical directions to minimize the total chip area while ensuring that all geometric features continue to satisfy the fabrication technology's design rules (e.g., minimum spacing and minimum width rules).

1.1 1D vs. 2D Compaction

- **1D Compaction:** Squeezes the layout in one direction at a time (first X, then Y, or vice versa). It is computationally simple ($O(n \log n)$ or $O(n^2)$) but can yield suboptimal results because vertical and horizontal constraints are decoupled.
- **2D Compaction:** Squeezes the layout in both X and Y directions simultaneously. It finds the absolute minimum area but is NP-complete, making it impractical for large layouts.

1.2 Constraint Graph Compaction

A layout is modeled as a set of features (contacts, wires, gates) whose coordinates are variables. Spacing constraints between feature i and feature j are represented as:

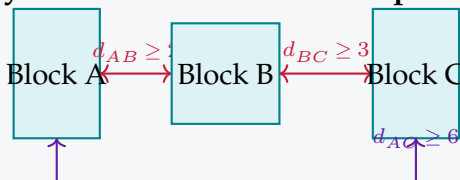
$$x_j - x_i \geq d_{ij} \quad (1)$$

where x_i, x_j are coordinates and d_{ij} is the minimum spacing requirement plus the widths of features.

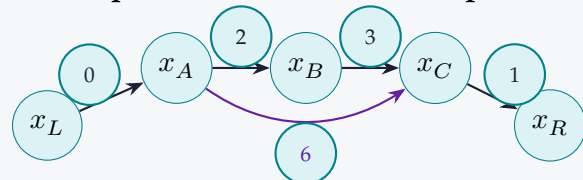
- **Graph Construction:** Vertices represent layout elements (coordinates). Directed edges represent constraints. An edge $e(i, j)$ with weight d_{ij} represents the constraint $x_j \geq x_i + d_{ij}$.
- **Longest Path formulation:** To find the minimum coordinates, we solve the longest path problem from a virtual source node ($x_{\text{left}} = 0$) using the Liao-Sarrafzadeh or Bellman-Ford algorithms.

Layout Compaction Spacing Constraints

Layout Blocks & Minimum Spacing:



Equivalent Constraint Graph:



2. Circuit Placement

2.1 1. Force-Directed Placement

Force-Directed Placement models the placement problem as a physical system of springs (Hooke's law). Standard cells are point masses, and interconnects are springs.

- **Spring Force Formula:** The force between cell i and cell j connected by a net with weight w_{ij} is:

$$F_{ij} = w_{ij}(x_j - x_i) \quad (2)$$

- **Equilibrium Equations:** The total force on cell i must be zero:

$$\sum_j w_{ij}(x_j - x_i) = 0 \quad \Rightarrow \quad x_i \sum_j w_{ij} - \sum_j w_{ij}x_j = 0 \quad (3)$$

This is written in matrix form as $Cx = f$, where C is the connectivity Laplacian matrix. Solving this system gives optimal cell coordinates that minimize quadratic wirelength.

2.2 2. Min-Cut Placement (Breuer's Algorithm)

Min-Cut placement uses a top-down divide-and-conquer strategy:

1. Start with the entire chip area containing all standard cells.
2. Bisect the chip area horizontally or vertically.
3. Use a graph partitioning algorithm (e.g. Kernighan-Lin) to partition the cells into two subsets, minimizing the number of nets crossing the cut line.
4. Recursively apply this bisection to the sub-regions until each region contains only one or a few cells.

3. Circuit Partitioning

Circuit Partitioning

Circuit Partitioning is the process of dividing a circuit (modeled as a graph or hypergraph) into two or more sub-circuits (partitions) such that the size of each partition is balanced (respecting capacity limits) and the number of connections between the partitions (the cut size) is minimized.

3.1 Kernighan-Lin (KL) Algorithm

The KL algorithm is an iterative, heuristic bipartitioning algorithm for undirected graphs. It starts with an initial balanced partition (A, B) of size n each ($|V| = 2n$), and swaps pairs of nodes to reduce the cut cost.

3.1.1 Mathematical Formulations

For each node $a \in A$:

- **External Cost (E_a):** Connection cost to nodes in the other partition: $E_a = \sum_{y \in B} c_{ay}$.

- **Internal Cost** (I_a): Connection cost to nodes in the same partition: $I_a = \sum_{x \in A} c_{ax}$.
- **D-Value** (D_a): Difference representing the priority of node a for swapping:

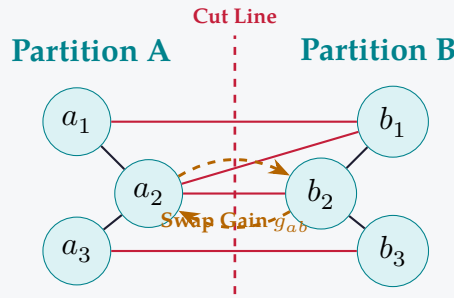
$$D_a = E_a - I_a \quad (4)$$

- **Gain** (g_{ab}): The net reduction in cut cost if node $a \in A$ and $b \in B$ are swapped:

$$g_{ab} = D_a + D_b - 2c_{ab} \quad (5)$$

where c_{ab} is the weight of the edge between a and b .

Kernighan-Lin Swapping Concept



Swap gain formula: $g_{ab} = D_a + D_b - 2c_{ab}$. If swapped, the cut size decreases by g_{ab} .

3.1.2 Algorithmic Steps for One Pass

1. Mark all vertices as *unlocked*.
2. Compute D -values for all vertices in A and B .
3. For $i = 1$ to n :
 - Choose an unlocked pair $(a_i, b_i) \in A \times B$ that maximizes $g_i = D_{a_i} + D_{b_i} - 2c_{a_i b_i}$.
 - Lock a_i and b_i (they cannot be swapped again in this pass).
 - Update D -values of remaining unlocked neighbors:
$$D'_x = D_x + 2c_{xa_i} - 2c_{xb_i} \quad \text{for } x \in A$$

$$D'_y = D_y + 2c_{yb_i} - 2c_{ya_i} \quad \text{for } y \in B$$
4. Find the step k that maximizes the cumulative gain $G_k = \sum_{i=1}^k g_i$.
5. If $G_k > 0$, swap the sets $\{a_1, \dots, a_k\}$ and $\{b_1, \dots, b_k\}$ permanently and start a new pass. If $G_k \leq 0$, terminate (no further improvement possible).

Kernighan-Lin Step-by-Step Example

Problem:

Consider a graph with $V = \{1, 2, 3, 4, 5, 6\}$ and edges $E = \{(1, 2), (1, 3), (1, 4), (2, 5), (3, 6), (4, 5), (5, 6)\}$. The initial partitions are $A = \{1, 2, 3\}$ and $B = \{4, 5, 6\}$. Find the gains and update the partitions.

Solution:

Step 1: Compute initial D-values:

- Node 1: $E_1 = c_{14} = 1; I_1 = c_{12} + c_{13} = 2. \Rightarrow D_1 = 1 - 2 = -1.$
- Node 2: $E_2 = c_{25} = 1; I_2 = c_{21} = 1. \Rightarrow D_2 = 1 - 1 = 0.$
- Node 3: $E_3 = c_{36} = 1; I_3 = c_{31} = 1. \Rightarrow D_3 = 1 - 1 = 0.$
- Node 4: $E_4 = c_{41} = 1; I_4 = c_{45} = 1. \Rightarrow D_4 = 1 - 1 = 0.$
- Node 5: $E_5 = c_{52} = 1; I_5 = c_{54} + c_{56} = 2. \Rightarrow D_5 = 1 - 2 = -1.$
- Node 6: $E_6 = c_{63} = 1; I_6 = c_{65} = 1. \Rightarrow D_6 = 1 - 1 = 0.$

Step 2: Calculate gains $g_{ij} = D_i + D_j - 2c_{ij}$ for unlocked pairs:

- $g_{14} = D_1 + D_4 - 2c_{14} = -1 + 0 - 2(1) = -3$
- $g_{15} = D_1 + D_5 - 2c_{15} = -1 - 1 - 0 = -2$
- $g_{24} = D_2 + D_4 - 2c_{24} = 0 + 0 - 0 = 0$
- $g_{25} = D_2 + D_5 - 2c_{25} = 0 - 1 - 2(1) = -3$
- $g_{36} = D_3 + D_6 - 2c_{36} = 0 + 0 - 2(1) = -2 \dots$

The maximum gain is $g_1 = g_{24} = 0$ (swap Node 2 and Node 4). Lock Node 2 and Node 4.

Step 3: Update D-values for remaining unlocked nodes $\{1, 3\}$ and $\{5, 6\}$:

- $D'_1 = D_1 + 2c_{12} - 2c_{14} = -1 + 2(1) - 2(1) = -1$
- $D'_3 = D_3 + 2c_{32} - 2c_{34} = 0 + 0 - 0 = 0$
- $D'_5 = D_5 + 2c_{54} - 2c_{52} = -1 + 2(1) - 2(1) = -1$
- $D'_6 = D_6 + 2c_{64} - 2c_{62} = 0 + 0 - 0 = 0$

Calculate gains for the second step:

- $g_{15} = D'_1 + D'_5 - 2c_{15} = -1 - 1 - 0 = -2$
- $g_{16} = D'_1 + D'_6 - 2c_{16} = -1 + 0 - 0 = -1$
- $g_{35} = D'_3 + D'_5 - 2c_{35} = 0 - 1 - 0 = -1$
- $g_{36} = D'_3 + D'_6 - 2c_{36} = 0 + 0 - 2(1) = -2$

Choose swap pair (1, 6) or (3, 5) with $g_2 = -1$. Let's select (3, 5) to swap. Lock $\{3, 5\}$. After calculating the remaining gain, we determine that cumulative gain G_k is not positive. The optimal choice is to not make any swaps, or restart with a different initial partition.

3.2 Fiduccia-Mattheyses (FM) Partitioning Algorithm

The FM algorithm is an extension of the KL algorithm that partitions hypergraphs. Key enhancements over KL:

- **Single Node Moves:** Instead of swapping pairs, it moves one cell at a time from its partition to another. This allows handling partitions of unequal sizes.
- **Linear Time Complexity:** Time complexity is reduced to $O(|E|)$ per pass by using a bucket

array data structure to select the unlocked cell with the maximum gain in $O(1)$ time.

4. Quick Revision Page

4.1 Key Equations & Formulations

Compaction, Placement & Partitioning Formulas

1. Liao-Sarrafzadeh Constraint Graph Spacing Constraint:

$$x_j - x_i \geq d_{ij} \quad (\text{represented as a directed edge } v_i \rightarrow v_j \text{ with weight } d_{ij})$$

2. Force-Directed Equilibrium Coordinate:

$$x_i = \frac{\sum_j w_{ij} x_j}{\sum_j w_{ij}}$$

Where w_{ij} is the connection weight (number of interconnect wires) between cell i and cell j .

3. Kernighan-Lin Gain (g_{ab}):

$$g_{ab} = D_a + D_b - 2c_{ab}$$

Where $D_a = E_a - I_a$ is the priority metric of node a , and c_{ab} is the connection weight between a and b .

4.2 One-Line Definitions

- **1D Compaction:** Compressing elements along one axis in a single step (either horizontal or vertical).
- **Constraint Graph:** A directed graph representing layout elements as vertices and spacing rules as weighted edges.
- **Hypergraph:** A graph where a hyperedge can connect any number of vertices, used to model multi-terminal nets.
- **Laplacian Matrix:** A matrix representation of connectivity used in force-directed placements.
- **Cut Size:** The number of nets or edges crossing the boundaries separating different partitions.
- **Gain (KL):** The net reduction in the cut cost resulting from swapping two nodes between partitions.
- **FM Algorithm:** A linear-time hypergraph partitioning algorithm that moves a single node at a time.

4.3 Comparison: Algorithms & Compaction

1D vs. 2D Compaction:

Feature	1D Compaction	2D Compaction
Squeezing Axis	Squeezes sequentially (X first, then Y).	Squeezes simultaneously in both X and Y.
Complexity	Polynomial time ($O(n \log n)$), very fast.	NP-complete, computationally intractable.
Optimality	Suboptimal (leaves redundant space).	Achieves absolute minimum silicon area.

KL vs. FM Partitioning:

Feature	Kernighan-Lin (KL) Algorithm	Fiduccia-Mattheyses (FM) Algorithm
Circuit Model	Standard graph (edges connect two nodes).	Hypergraph (hyperedges connect multiple nodes).
Move Strategy	Swaps pairs of nodes ($a \in A, b \in B$).	Moves a single node at a time.
Time Complexity	$O(n^3)$ per pass.	$O(E)$ (linear time) per pass.
Partition Size	Forces strictly equal sizes.	Supports flexible/unbalanced partition sizes.

4.4 Frequently Asked Questions / Viva Questions

Exam Q&A

1. What are spacing and width rules in layout compaction?

Width rules dictate the minimum dimension of a layout wire or layer (to prevent breaks due to line width tolerances). Spacing rules define the minimum distance between two separate features (to prevent short circuits due to lithographic misalignment).

2. How is a layout feature mapped to a constraint graph during compaction?

Each vertical layout edge (for X-compaction) or horizontal layout edge (for Y-compaction) is represented as a vertex. The minimum design rule spacing between the two edges is represented as a directed, weighted edge connecting the vertices.

3. What is the difference between a graph and a hypergraph?

In a standard graph, each edge connects exactly two vertices. In a hypergraph, an edge (hyperedge) can connect a subset containing any number of vertices. Circuits are modeled as hypergraphs because a single signal line (net) typically connects multiple component pins.

4. Why does the KL partitioning algorithm lock swapped nodes?

If swapped nodes were not locked, the algorithm could end up in an infinite loop by swapping the same nodes back and forth. Locking them guarantees that every node is swapped at most once per pass, ensuring termination in exactly n steps.

5. What is the purpose of the virtual source and sink nodes in constraint graphs?

The virtual source node (x_{left}) represents the left edge of the chip boundary ($x = 0$). The virtual sink node (x_{right}) represents the right boundary. The longest path from source to sink determines the minimum width of the entire compacted layout.

6. How does force-directed placement handle cell overlaps?

Force-directed placement minimizes total quadratic wirelength, which pulls all connected cells toward the center of the chip, causing significant cell overlaps. To resolve overlaps, placement tools follow the force calculation step with slot assignment or recursive partitioning.

7. Explain the role of the bucket array structure in the FM algorithm.

The FM algorithm uses a bucket array to index unlocked cells by their gain values. Cells with the same gain are linked together. This data structure allows the cell with the maximum gain to be found and retrieved in $O(1)$ constant time, enabling overall linear time complexity.

8. What are the limitations of the KL algorithm?

(1) It only works on standard graphs, not hypergraphs. (2) It can only handle partitions of equal sizes. (3) Its time complexity is $O(n^3)$ per pass, which is too slow for modern circuits containing millions of nodes.

Frequently Asked / Viva Questions (contd.)**9. Why is quadratic wirelength used in force-directed placement instead of linear wirelength?**

Quadratic wirelength uses a squared distance metric (d^2), which can be formulated as a set of linear equations. Solving linear equations is mathematically straightforward and fast (using matrix solvers). Linear wirelength optimization requires absolute values ($|d|$), which cannot be solved directly in a single step.

10. What is a connectivity matrix, and how does it represent net weights?

A connectivity matrix C is a matrix where the entry C_{ij} represents the number of nets connecting cell i and cell j . Higher values of C_{ij} indicate a stronger connection, which generates a stronger attractive force during force-directed placement, pulling the cells closer together.